

Reviewer Pack — Minimal Geometric Invariant of Polyhedral Sections and Commu...

Entry 82e1463096c9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 19:17:24 UTC

Minimal Geometric Invariant of Polyhedral Sections and Communication Complexity Lower Bounds

Entry ID: 82e1463096c9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 19:17:24 UTC

1. Conjecture

Field A (mathematical branch): Convex Geometry **Field B** (complexity object): Communication Complexity

Statement:

[‘For any polyhedron P with facets bounded by simplices, the minimum geometric invariant of a hyperplane section of P is at least $\Omega(n \log n)$, where n is the number of bits in the description of P .’, ‘This geometric invariant is defined as the minimum distance between any point on the boundary of the section and the hyperplane.’, ‘For all instances with property Q , the communication complexity of a function f is lower bounded by $O(\log^2(n))$.’]

Rationale (proposer’s reasoning):

[“Convex geometry provides a framework for studying the geometric properties of polyhedra. The minimal geometric invariant of a section captures its ‘size’ and should correlate with the communication complexity required to communicate the information.”, ‘Communication complexity measures the amount of information that needs to be exchanged between two parties to compute a function, which is analogous to the distance between points in geometry.’]

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 25688ac47ce26a7d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the minimum geometric invariant for at least 90% of polyhedra P is greater than or equal to $\Omega(n \log n)$, and for at least 90% of instances with property Q , the communication complexity of function f is lower bounded by $O(\log^2(n))$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.80	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (2): - "polyhedral sections" AND "geometric invariant" AND convex geometry" - "communication complexity" AND "lower bounds" AND polyhedral sections", "convex geometry" AND "hyperplane section" AND communication complexity

Top relevant hits considered: - [<http://arxiv.org/abs/2210.01601v2>] Quantum communication complexity of linear regression - [<http://arxiv.org/abs/1709.04262v2>] Lower Bounds for Approximating Graph Parameters via Communication Complexity - [<http://arxiv.org/abs/1007.1841v1>] Communication Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below
        for j in range(i+1, n):
            factor = Fraction(matrix[j][i], matrix[i][i])
            for k in range(n+1):
                matrix[j][k] -= factor * matrix[i][k]

    # Back substitution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = Fraction(matrix[i][n], matrix[i][i])
        for j in range(i-1, -1, -1):
```

```

        matrix[j][n] -= matrix[j][i] * x[i]
    return x

def solve_linear_system(A, b):
    augmented_matrix = [row + [b[i]] for i, row in enumerate(A)]
    solution = gaussian_elimination(augmented_matrix)
    return solution

def min_geometric_invariant(vertices, facets):
    n = len(vertices[0])
    min_dist = float('inf')

    for facet in facets:
        A = []
        b = []
        for vertex in vertices:
            eq = [vertex[i] for i in facet]
            A.append(eq)
            b.append(-eq[facet[-1]])

        try:
            solution = solve_linear_system(A, b)
            dist = sum(x**2 for x in solution) ** 0.5
            if dist < min_dist:
                min_dist = dist
        except Exception as e:
            return float('inf')

    return min_dist

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    vertices = [[random.uniform(-1, 1) for _ in range(n)] for _ in range(random.randint(3, n+2))]
    facets = [sorted(random.sample(range(n), k=random.randint(2, n))) for _ in range(random.randint(1, n))]

    min_dist = min_geometric_invariant(vertices, facets)
    communication_complexity = math.log2(n) ** 2

    return {
        "metric_name": "min_geometric_invariant",
        "metric_value": min_dist,
        "instances_tested": len(facets),
        "conjecture_holds": min_dist >= n * math.log2(n) and communication_complexity <= (n * math.log2(n)) ** 2,
        "counterexample": "" if min_dist >= n * math.log2(n) else f"min_geometric_invariant < {n * math.log2(n)}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

```

```

mean_metric = sum(r['metric_value'] for r in results) / len(results)
std_metric = (sum((r['metric_value'] - mean_metric) ** 2 for r in results) / len(results)) ** 0.5
support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

if all(r['conjecture_holds'] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
elif support_fraction >= 0.9:
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
    print(f"RESULT: FALSIFIED counterexample=\"min_geometric_invariant < {n * math.log2(n)}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_23da22cb.py", line 92, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_23da22cb.py", line 75, in run_trial
    min_dist = min_geometric_invariant(vertices, facets)
                ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_23da22cb.py", line 57, in min_geometric_invariant
    b.append(-eq[facet[-1]])
              ~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test without errors to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14481
2	preregistration	ollama_remote	glm4:latest	0	0	9463

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	glm4:latest	0	0	8473
4	novelty	ollama_remote	glm4:latest	0	0	9734
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18851
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9344
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10742
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12029
9	judge	ollama_remote	glm4:latest	0	0	11509

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 104626 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/82e1463096c9.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/82e1463096c9.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/82e1463096c9.tar.gz* (if generated)